Właściwości

Właściwości

Przedstawione podejście do enkapsulacji (niepubliczne pola, publiczne wybrane metody) jest typowe dla wielu obiektowych języków programowania (np. Java, C++). W języku Python często pola są udostępniane jako składowe publiczne. Jest to spowodowane tym, że mechanizm **właściwości** (ang. *properties*) umożliwia w razie potrzeby bezproblemową zmianę odwołania do pola na wywołanie metody.

Jeżeli składowa jest właściwością to odwołanie do niej składniowo wygląda jak odwołanie do pola (odczyt wartości pola, podstawienie pod pole), ale realizowane jest przez wywołanie odpowiedniej metody. Poniższy przykład pokazuje implementację klasy `Kolo` z użyciem właściwości:

In [2]:
```python
import math


class Kolo:
    def __init__(self, promien):
        self.promien = promien

    def _sprawdz_promien(self, promien):
        if not isinstance(promien, (int, float)):
            raise TypeError("promień musi być liczbą")
        if promien <= 0:
            raise ValueError("promień musi być dodatni")

    @property
    def promien(self):
        return self._promien

    @promien.setter
    def promien(self, promien):
        self._sprawdz_promien(promien)
        self._promien = promien

    @property
    def obwod(self):
        return 2 * math.pi * self._promien

    @property
    def pole(self):
        return math.pi * self._promien**2
```

Składowe `promien`, `obwod` i `pole` są właściwościami, których wartości możemy odczytać. Wywoływane są wtedy odpowiednie metody (nazwane tak jak właściwość i oznaczone tzw. dekoratorem `@property`).

In [3]:
```
k = Kolo(3.0)
```

In [4]:
```
k.promien
```

Out[4]: 3.0

In [5]:
```
k.obwod
```

Out[5]: 18.84955592153876

In [6]:
```
k.pole
```

Out[6]: 28.274333882308138

Właściwość `promien` możemy również modyfikować (zostanie wywołana odpowiednia funkcja — również nazwana tak jak właściwość i oznaczona dekoratorem `@promien.setter`):

In [7]:
```
k.promien = 6.0
```

In [8]:
```
k.promien
```

Out[8]: 6.0

In [9]:
```
k.obwod
```

Out[9]: 37.69911184307752

In [10]:
```
k.pole
```

Out[10]: 113.09733552923255

Setter właściwości `promien` sprawdza typ i wartość przekazanego argumentu. Próba przypisania napisu zamiast liczby spowoduje wyjątek `TypeError`:

In [11]:
```
k.promien = "34"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[11], line 1
----> 1 k.promien = "34"

Cell In[2], line 20, in Kolo.promien(self, promien)
     18     @promien.setter
     19     def promien(self, promien):
---> 20         self._sprawdz_promien(promien)
     21         self._promien = promien

Cell In[2], line 10, in Kolo._sprawdz_promien(self, promien)
      8     def _sprawdz_promien(self, promien):
      9         if not isinstance(promien, (int, float)):
---> 10             raise TypeError("promień musi być liczbą")
     11         if promien <= 0:
     12             raise ValueError("promień musi być dodatni")

TypeError: promień musi być liczbą
```

Ujemna liczba ma odpowiedni typ, ale nie jest poprawnym promieniem. Próba przypisania jej do właściwości `promien` spowoduje wyjątek `ValueError`:

In [12]:
```
k.promien = -4
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[12], line 1
----> 1 k.promien = -4

Cell In[2], line 20, in Kolo.promien(self, promien)
     18     @promien.setter
     19     def promien(self, promien):
---> 20         self._sprawdz_promien(promien)
     21         self._promien = promien

Cell In[2], line 12, in Kolo._sprawdz_promien(self, promien)
      8     def _sprawdz_promien(self, promien):
      9         if not isinstance(promien, (int, float)):
     10             raise TypeError("promień musi być liczbą")
     11         if promien <= 0:
---> 12             raise ValueError("promień musi być dodatni")

ValueError: promień musi być dodatni
```

Właściwości bez zdefiniowanego settera są tylko do odczytu. W tej klasie taką właściwością jest `obwod`. Próba przypisania jej wartości spowoduje wyjątek `AttributeError`:

In [13]:
```
k.obwod = 10
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[13], line 1
----> 1 k.obwod = 10

AttributeError: property 'obwod' of 'Kolo' object has no setter
```

Właściwość `pole` również nie ma settera. Poniższa instrukcja podstawienia spowoduje z tego samego powodu wyjątek `AttributeError`:

In [14]:
```
k.pole = 5
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[14], line 1
----> 1 k.pole = 5

AttributeError: property 'pole' of 'Kolo' object has no setter
```

Instrukcja `self.promien = promien` w konstruktorze oznacza podstawienie pod właściwość `promien`, a nie podstawienie pod pole. Powoduje więc wywołanie settera i sprawdzenie poprawności promienia także podczas tworzenia obiektu. Próba utworzenia koła z ujemnym promieniem spowoduje wyjątek `ValueError`:

In [15]:
```
p = Kolo(-3.0)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[15], line 1
----> 1 p = Kolo(-3.0)

Cell In[2], line 6, in Kolo.__init__(self, promien)
      5     def __init__(self, promien):
----> 6         self.promien = promien

Cell In[2], line 20, in Kolo.promien(self, promien)
     18     @promien.setter
     19     def promien(self, promien):
---> 20         self._sprawdz_promien(promien)
     21         self._promien = promien

Cell In[2], line 12, in Kolo._sprawdz_promien(self, promien)
      8     def _sprawdz_promien(self, promien):
      9         if not isinstance(promien, (int, float)):
     10             raise TypeError("promień musi być liczbą")
     11         if promien <= 0:
---> 12             raise ValueError("promień musi być dodatni")

ValueError: promień musi być dodatni
```

Klasa `Konto` zaimplementowana z użyciem właściwości mogłaby wyglądać tak:

In [16]:

```python
class Konto:
    def __init__(self, nr_konta):
        self._nr_konta = nr_konta
        self._stan = 0
        self._dopuszczalny_debet = 0

    def _sprawdz_kwote(self, kwota):
        if not isinstance(kwota, int):
            raise TypeError("kwota musi być typu int")
        if kwota <= 0:
            raise ValueError("kwota musi być dodatnia")

    def _log(self, komunikat):
        with open(
            f"{self._nr_konta}_log.txt", "a", encoding="utf-8"
        ) as log:
            print(komunikat, file=log)
            print(f"stan konta po operacji: {self._stan}", file=log)
            print("---", file=log)

    @property
    def stan(self):
        self._log("sprawdzenie stanu")
        return self._stan

    @property
    def dopuszczalny_debet(self):
        self._log("sprawdzenie dopuszczalnego debetu")
        return self._dopuszczalny_debet

    @dopuszczalny_debet.setter
    def dopuszczalny_debet(self, dopuszczalny_debet):
        if not isinstance(dopuszczalny_debet, int):
            raise TypeError("debet musi być typu int")
        if dopuszczalny_debet < 0 or self._stan < -dopuszczalny_debet:
            raise ValueError("niedopuszczalny debet")
        self._log(
            f"ustawienie dopuszczalnego debetu na {dopuszczalny_debet}"
        )
        self._dopuszczalny_debet = dopuszczalny_debet

    @property
    def wolne_srodki(self):
        self._log("sprawdzenie wolnych środków")
        return self._stan + self._dopuszczalny_debet

    def wplata(self, kwota):
        self._sprawdz_kwote(kwota)
        self._stan += kwota
        self._log(f"wpłata: {kwota}")

    def wyplata(self, kwota):
        self._sprawdz_kwote(kwota)
        if kwota > self.wolne_srodki:
            raise ValueError("brak środków")
```

```
        self._stan -= kwota
        self._log(f"wypłata: {kwota}")
```

In [17]:
```
k = Konto("12345")
```

In [18]:
```
k.stan
```

Out[18]:
```
0
```

Setter właściwości `dopuszczalny_debet` również sprawdza przekazaną wartość. W poniższej instrukcji użyto napisu zamiast liczby całkowitej, co spowoduje wyjątek `TypeError`:

In [19]:
```
k.dopuszczalny_debet = "5000"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[19], line 1
----> 1 k.dopuszczalny_debet = "5000"

Cell In[16], line 32, in Konto.dopuszczalny_debet(self, dopuszczalny_debet)
     29     @dopuszczalny_debet.setter
     30     def dopuszczalny_debet(self, dopuszczalny_debet):
     31         if not isinstance(dopuszczalny_debet, int):
---> 32             raise TypeError("debet musi być typu int")
     33         if dopuszczalny_debet < 0 or self._stan < -dopuszczalny_debe
t:
     34             raise ValueError("niedopuszczalny debet")
     35         self._log(f"ustawienie dopuszczalnego debetu na {dopuszczaln
y_debet}")

TypeError: debet musi być typu int
```

Dopuszczalny debet nie może być ujemny. Próba przypisania poniższej wartości do właściwości `dopuszczalny_debet` spowoduje wyjątek `ValueError`:

In [20]:
```
k.dopuszczalny_debet = -4000
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[20], line 1
----> 1 k.dopuszczalny_debet = -4000

Cell In[16], line 34, in Konto.dopuszczalny_debet(self, dopuszczalny_debet)
     30     def dopuszczalny_debet(self, dopuszczalny_debet):
     31         if not isinstance(dopuszczalny_debet, int):
     32             raise TypeError("debet musi być typu int")
     33         if dopuszczalny_debet < 0 or self._stan < -dopuszczalny_debe
t:
---> 34             raise ValueError("niedopuszczalny debet")
     35         self._log(f"ustawienie dopuszczalnego debetu na {dopuszczaln
y_debet}")
     36         self._dopuszczalny_debet = dopuszczalny_debet

ValueError: niedopuszczalny debet
```

In [21]:
```
k.dopuszczalny_debet = 3000
```

In [22]:
```
k.wplata(1000)
```

In [23]:
```
k.wolne_srodki
```

Out[23]: 4000